SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

This section "Cook.GPT " is introducing the strategic solution to the problem of domestic food waste and financial leakage in Malaysian households. Unlike traditional recipe apps, Cook.GPT is an economic decision engine that converts visual pantry data into actionable financial savings.

1.1. Purpose:

*The system analysis document highlights the technical scope and design-related decisions behind the Cook.GPT development. It serves to align the **KuliahF3** developers on how computer vision and financial logic integrate to solve the user's pain points.*

So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
    a. *Cook.GPT uses Client/Server architecture optimized for mobile-first performance.*
    b. *Frontend is built with React (Vite).*
    c. *AI Engine is powered by Z.AI (ILMU-GLM-5.1), which functions as a centralized intelligence hub for both image recognition and recipe synthesis.*
    d. *Deployment is managed via the hackathon's dedicated server infrastructure with a Claude Code integrated pipeline for rapid iteration.*
2. ***Data Flows:***
    a. *The user captures an image of their fridge.*
    b. *The frontend converts this into a Base64 string and POSTs it to our backend API.*
    c. *The Z.AI model parses the image to extract a list of ingredients.*
    d. *These are then passed through our Economic Logic Layer, which assigns a "Risk Value" based on perishability.*
    e. *Finally, a structured JSON object containing recipes and RM savings data is returned to the Reach frontend for dashboard visualization.*
3. ***Model Process:***
    a. ***Identify:** Multi-object detection of Malaysian-specific ingredients (e.g., Cili Padi, Lemongrass).*
    b. ***Quantify:** Calculate the "Total Savable RM" by cross-referencing detected items with local market price estimates.*
    c. ***Rescue:** Generate "Smart Substitution" recipes that prioritize the highest-risk (expiring soonest) ingredients to ensure zero-waste.*
4. ***Role of Reference:***
    a. *This document serves as the "Single Source of Truth" for the **Team KuliahF3** programmer and designer. It ensures that the UI components (Impact Tracker) align perfectly with the backend data structures (JSON schemas) provided by the AI.*

© UMHackathon 2026

1.2. *Background:*

Team KuliahF3 *recognized that while Malaysia has a vibrant food culture, it also faces rising grocery costs. We identified a gap where households have the "assets" (food) but lack the "intelligence" to manage them before they depreciate to zero value (expire).*

1. *Previous Version:*
    a. ***Manual Apps:*** *Require users to type in every onion and garlic clove, leading to high user churn.*
    b. ***Static Books:*** *Suggest recipes that often require more shopping, which defeats the purpose of saving money.*
2. *Changes in Major Architectural Components:*
    a. *Cook.GPT moves away from the "Search" paradigm to the* ***"Inference"*** *paradigm.*
3. *New capabilities that are only introduced in this:*
    a. ***Dynamic Ingredient Substitution:*** *The AI can "think" like a chef, swapping ingredients based only on what is visually present in the user's actual inventory.*
    b. ***Gamified Impact:*** *Transforming mundane cooking into a "Level Up" experience by tracking cumulative RM savings.*

1.3. *Target Stakeholder*

Stakeholders	***Roles***	***Expectations***
Customer	*Uploads pantry photos, reviews financial "Rescue" metrics, and follows generated recipes.*	*Zero manual data entry; high accuracy in ingredient detection; recipes that taste good.*
Development Team	*Integrates* ***Z.AI Vision*** *and* ***ILMU-GLM-5.1****; manages the JSON-to-UI data pipeline.*	*Robust error handling for non-food images; low latency between photo upload and recipe output.*
QA Team	*Validates "Decision Intelligence" by testing the AI's substitution logic and price calculations.*	*Accuracy in RM value calculation; ensuring "Smart Substitutions" don't suggest dangerous or inedible combinations.*
Local Grocery Partners (Future)	*(Strategic Growth) Providing API access to real-time price data for more accurate RM savings.*	*Direct conversion from "missing ingredient" to a "one-click purchase" in their digital storefront.*

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

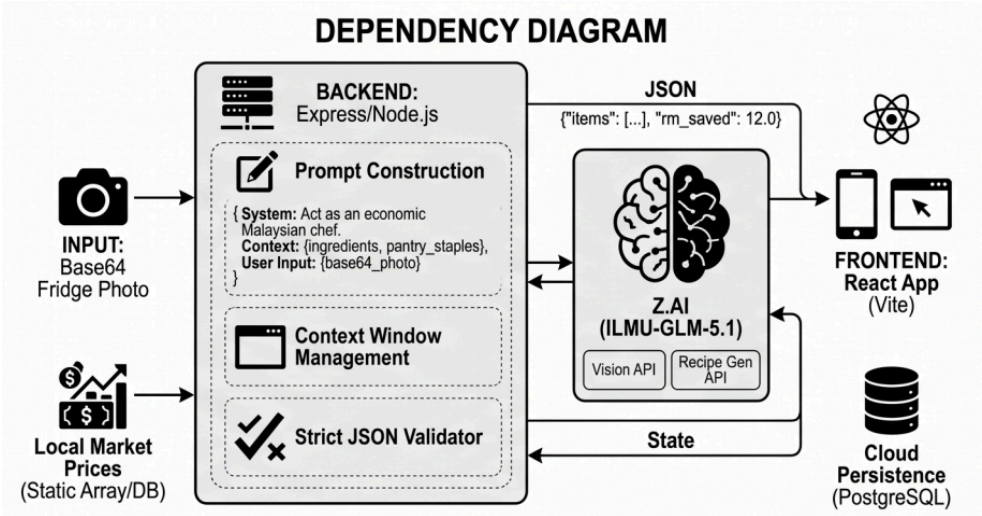
Type	Details
System	*Mobile-First Web Application (React.js / Vite)*
Architecture	*Client-Server Architecture with Serverless Data and AI Integration*

The system is developed as a Mobile-First Web Application utilizing React.js and Vite. This approach ensures rapid client-side rendering and high performance while providing a responsive, native-app-like user experience across different devices.

2.1.2 LLM as Service Layer

Cook.GPT integrates the **Z.AI (ILMU-GLM-5.1)** model not as a simple chatbot, but as a structured **Inference Engine**. This layer is responsible for translating unstructured visual "chaos" (a messy fridge) into structured financial and culinary data.

2.1.3 Dependency Diagram

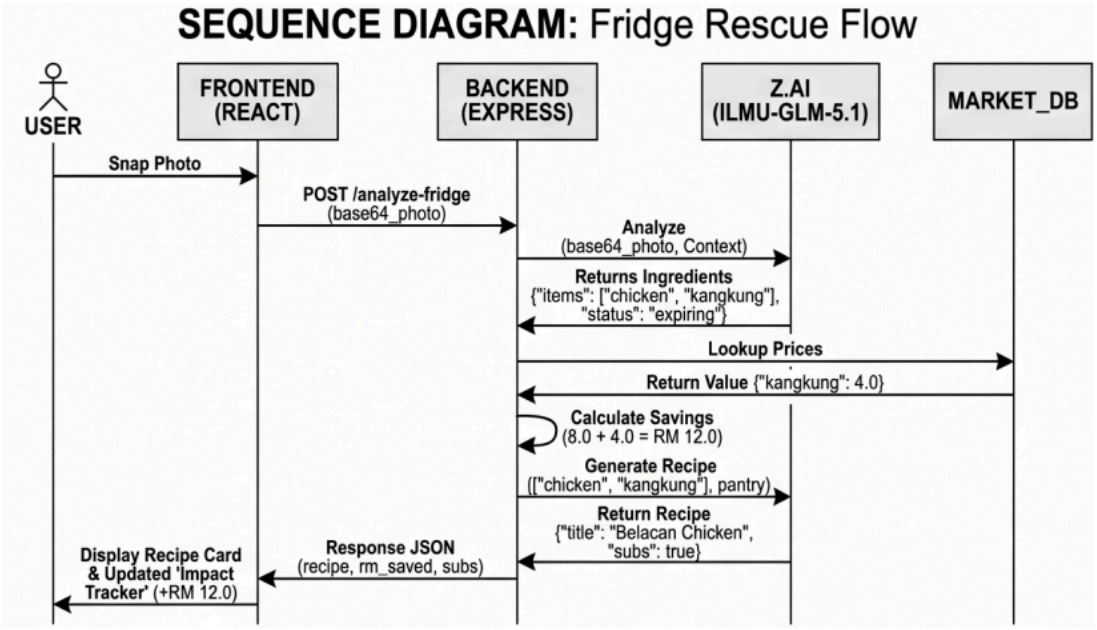


© UMHackathon 2026

b. Major API Interactions

1. ***Frontend*** → ***Backend:*** *POST /analyze-fridge (Payload: Image string).*
2. ***Backend*** → ***Z.AI API:*** *POST /v1/chat/completions (System Prompt: "Act as a Malaysian Zero-Waste Chef").*
3. ***Backend*** → ***Database:*** *UPDATE user_stats (Increment total_rm_saved based on AI output).*

2.2.2. Sequence Diagram



2.2. Technological Stack

2.2.1. Frontend: React (Vite) *– Optimized for mobile-first speed and fast "Last Minute" responsiveness.*

2.2.2. Backend: Node.js (Express) *or* ***FastAPI*** *– Chosen for seamless JSON handling and integration with Z.AI SDKs.*

2.2.3. Database: PostgreSQL (Supabase/RDS) *– To store the "History of Savings" and User XP.*

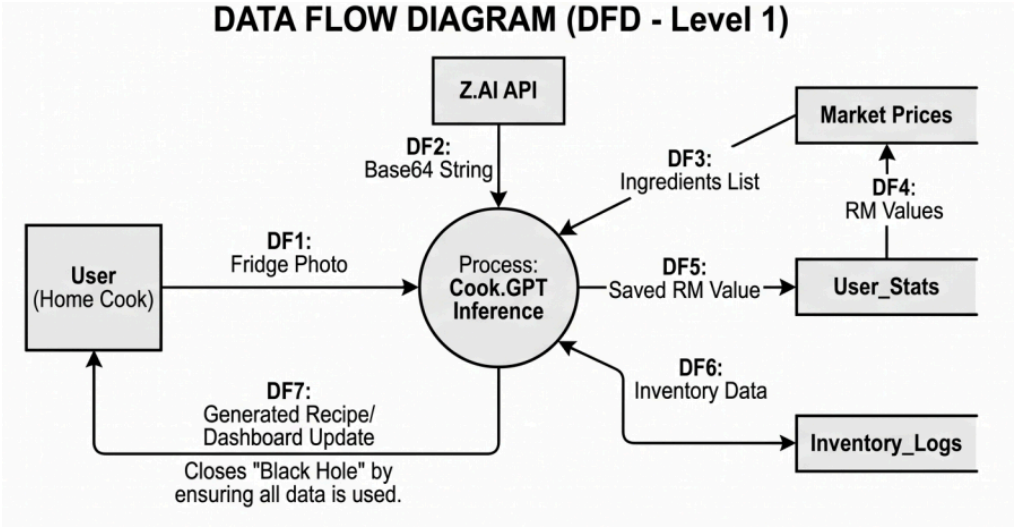
2.2.4. AI Engine: Z.AI (ILMU-GLM-5.1) *– Multi-modal (Vision + Text) capabilities.*

2.2.5. Dev Tools: Claude Code *– For rapid prototyping and automated unit testing of the JSON schemas.*

© UMHackathon 2026

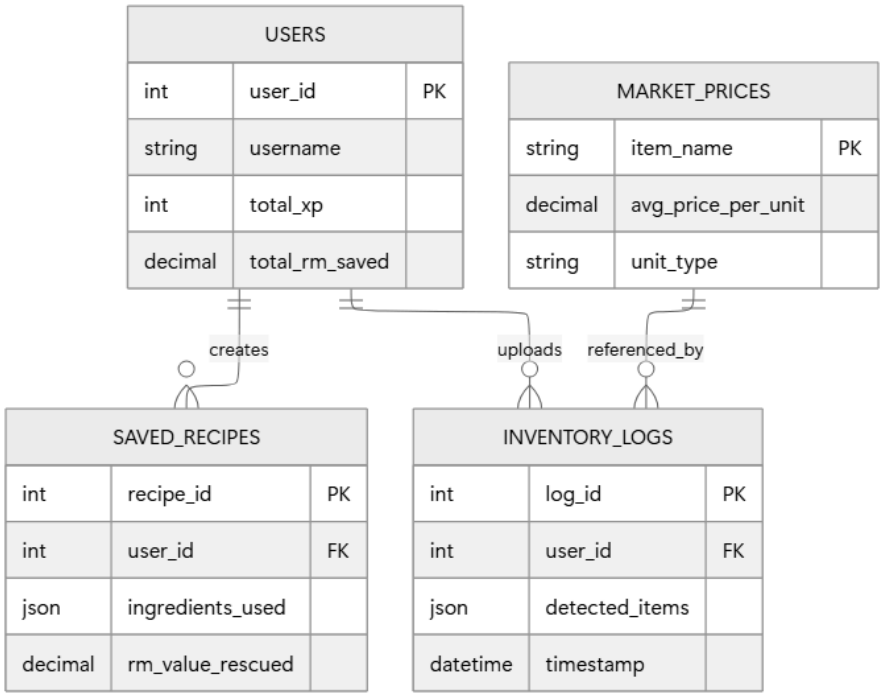
2.3. Key Data Flows

2.3.1. Data Flow Diagram (DFD)



2.3.2. Normalized Database Schema

To ensure data integrity, we use a 3NF structure for the financial tracking:



© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

#	Features	Description
1	Visual Pantry Audit	The user snaps a photo; the **Z.AI Vision** engine identifies ingredients (e.g., Cili Padi, Ayam, Sawi) with zero manual typing.
2	Economic Impact Logic	The system cross-references detected items with a local price matrix to calculate the **"Total Savable RM"** in the fridge.
3	Inference Recipe Engine	AI generates 2–3 culturally authentic Malaysian recipes using **only** what was seen in the photo + basic pantry staples.
4	Smart Substitution UI	Displays recipes with "Substitution Badges" (e.g., using Milk + Vinegar because Santan was missing) to show real-time problem solving.
5	Impact Dashboard	A gamified tracker showing "Total RM Saved" and "Waste Avoided (kg)" to visualize the economic benefit over time.

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g)
Inference Latency	The visual analysis and recipe generation must return a response within **4 seconds** at p95.	Use **Async/Await** for API calls; show a "Chef is thinking..." skeleton loader to manage perceived wait time.

© UMHackathon 2026

Cost Efficiency	*Average token cost per user session must stay below **RM 0.05**.*	*Downscale images to **512x512** before upload; use system prompts that enforce concise, structured JSON outputs.*
Accuracy (Precision)	*Ingredient detection should have a **>85% accuracy** for common Malaysian household staples.*	*Use **Few-Shot Prompting** in the LLM layer to tune the model specifically for Southeast Asian produce.*
Reliability	*"Rescue" recipes must be edible; the system should not suggest dangerous combinations (e.g., mixing bleach with food).*	*Implement **Content Safety Guardrails** in the Z.AI system prompt to filter out non-food items detected in photos.*
Mobile-First UX	*The web interface must be fully responsive and functional on mid-range smartphones common in Malaysia.*	*Use **React/Vite** with Tailwind CSS for lightweight, high-performance mobile rendering.*

3.3. Out of Scope / Future Enhancements

These features are part of the "Grand Vision" for scaling Cook.GPT beyond the hackathon.

a. ***Real-time Grocery Integration:*** *Direct API link to Jaya Grocer or Lotus's to "one-click buy" missing ingredients for a specific recipe.*
b. ***IoT Smart Fridge Sync:*** *Continuous monitoring of fridge contents via internal cameras for proactive "Expiry Alerts."*
c. ***Social & Community Sharing:*** *A social layer where users can share their "Zero-Waste" creations and compete in "Saving Leaderboards."*
d. ***Hyper-Personalized Nutrition:*** *Integrating health data (e.g., Apple Health/Google Fit) to suggest recipes based on the user's daily caloric needs.*
e. ***Weekly Waste Report:*** *A push notification or in-app summary every week: "This week you saved RM45.00. You let RM3.50 of spinach expire."*
f. ***WhatsApp/Telegram Sharing:*** *One-tap share of a recipe card (image + link)*
g. ***Halal / Dietary Filters:*** *Before generating recipes, let users tag: halal-only, no pork, no shellfish, vegetarian, nut-free.*

© UMHackathon 2026

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

a. ***Grayscale Rollout**: The "Smart Substitution" engine is initially released to **10% of users**. We monitor the **"Recipe Acceptance Rate"** (how many users actually cook the suggested meal). If the error rate in ingredient detection remains below **5%**, we scale to 100%.*

b. ***A/B Testing:** We test two "Decision Strategies":*
   - ***Variant A:** Prioritize recipes based on the **Highest RM Value** saved.*
   - ***Variant B:** Prioritize recipes based on the **Shortest Preparation Time**.*
   - ***Success Metric:** The variant that leads to the highest cumulative "RM Saved" in the user dashboard wins.*

4.1.2. Emergency Rollback (ER) & Golden Release:

a. ***ER Trigger:** An automated health check detects a **>2% JSON Schema Validation failure**.*

b. ***Action:** The pipeline immediately reverts the backend to the **"Golden Release"**—a stable version that uses a more conservative prompt or a fallback static recipe cache to ensure the app doesn't crash.*

4.1.3. Priority Matrix:

Priority	*Trigger Condition*	*Action / Mitigation*
P1: Critical	*Z.AI API is down or Latency > 15s.*	*Trigger "Graceful Degradation": Disable photo uploads and show a "Pantry Staples" static recipe list.*
P2: High	*Ingredient Detection Accuracy falls below 70%.*	*Revert to the previous System Prompt version and clear the AI cache.*
P3: Medium	*"Total RM Saved" fails to update in the DB.*	*Queue the updates in a local cache and retry the write operation every 5 minutes.*
P4: Low	*Minor UI misalignment on specific mobile screens.*	*Log the bug in the backlog for the next sprint/update.*

© UMHackathon 2026

4.2. *Monitoring:*

*We utilize a **Centralized Observability Stack** to monitor the "Health of the Brain":*

- ***Inference Monitoring:** Real-time logging of **Z.AI token usage** and response times. If the "Token-to-Recipe" time exceeds 5 seconds, an alert is sent to the dev team.*
- ***Validation Watchdog:** A middleware service that monitors the **JSON Validator**. If it catches an "Inedible Recipe" (e.g., suggesting a non-food item), it flags the log for manual prompt tuning.*

4.3. *Assumptions:*

a. ***Visual Clarity**: Users will provide photos with "reasonable" lighting where at least 50% of the ingredient is visible.*
b. ***Market Price Stability**: We assume the local market price data (e.g., RM 14/kg for chicken) remains valid for at least 7 days before needing an update.*
c. ***Smartphone Capability:** Users have devices capable of processing a **Base64 image conversion** in-browser without significant lag.*

4.4. *External Dependencies:*

Tools	*Purpose*	*Risks*
Z Ai GLM API	*Core Inference Engine (Vision + Reasoning).*	*High: API downtime or rate limits.*
PostgreSQL (Supabase)	*Persistent storage for User XP and Savings.*	*Medium: Database connection limit*
Claude Code	*Rapid deployment and code auditing.*	*Low: Tooling downtime during the hackathonteam*
Pricing API	*Fetching current Malaysian grocery prices.*	*Medium: Stale data or API failure.*

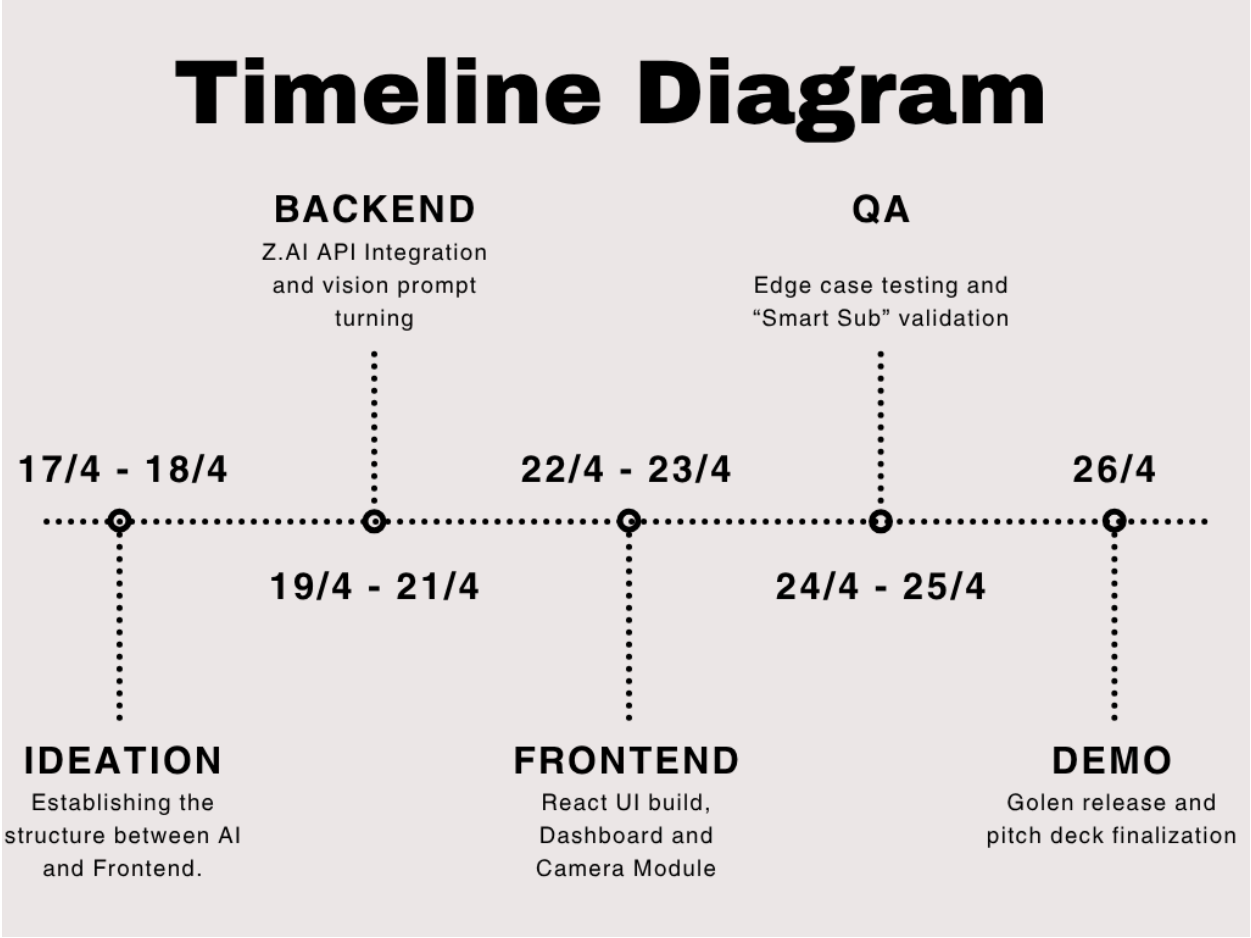
© UMHackathon 2026

Email: umhackathon@um.edu.my

5. *Project Management & Team Contributions*

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. *Project Timeline:*



5.2. *Team Members Role:*

a. *Howard Woon Hao Zhe - Project Lead - High-level system architecture design, implementation of frontend*
b. *Yim Zi Hao - Chief Strategy Officer - Drafting of documents, QA testing*
c. *Sanjay Mukojima Ravindran - Product Evangelist - Production of pitch video and live demo*
d. *Kingston Nuing Scott - Strategic Designer - Development of presentation pitch deck*

5.3. *Recommendations:*

a. ***Use Redis to cache "Inference Results"***

© UMHackathon 2026

If multiple users upload photos of a similar "Standard Malaysian Fridge", we can return a cached recipe instead of calling Z.AI API again, reducing latency and cost.

b. ***Implement a "Pantry-Staple" local database.***

If the Z.AI API hits a rate limit, the system should fail-safe by allowing the user to select 3 items manually from a list to generate a pre-cached "Rescue Recipe".

c. ***Client-side image compression using Canvas API***

By compressing the 5MB fridge photo to <500KB on the user's phone before it hits our server, we significantly reduce upload times and cloud storage costs.

© UMHackathon 2026